```
import akka.kafka.Subscriptions

object ConsumerPlainSource {
  ...

    Consumer
      .plainSource(consumerSettings, Subscriptions.topics("test"))
      .map { msg: ConsumerRecord[String, String] =>                ❶
        println(s"key = ${msg.key},
                  value = ${msg.value},
                  offset = ${msg.offset}")
      }
      .runWith(Sink.ignore)                                        ❷
  ...
}
```

❶ Gets consumer records from the source and prints them

❷ Consumes and discards the elements

You can put all this together to make the application shown in the following listing.

Listing 14.1 Basic example of a Alpakka Kafka consumer

```
import akka.actor.typed.ActorSystem
import akka.actor.typed.scaladsl.Behaviors
import akka.kafka.{ ConsumerSettings, Subscriptions }
import akka.kafka.scaladsl.Consumer
import akka.stream.scaladsl.Sink
import org.apache.kafka.clients.consumer.ConsumerConfig
import org.apache.kafka.common.serialization.StringDeserializer
import scala.io.StdIn

object ConsumerPlainSource {

  def main(args: Array[String]) = {

    implicit val system = ActorSystem(Behaviors.empty, "consumerOne")   ❶

    val config =
      system.settings.config.getConfig("akka.kafka.consumer")           ❷

    val consumerSettings: ConsumerSettings =                            ❸
      ConsumerSettings(
        config,
```